PortShare

How Tunneling Works

A practical explanation of the main tunneling logic: how a local port becomes a public HTTPS URL, how requests travel through the system, and what is permanent vs temporary.

Audience: engineers reading the monorepo. Based on the current PortShare code (Go server, desktop Electron client, CLI).

1. What PortShare is

PortShare is a reverse tunnel system. You run an app on localhost (for example port 3000). A small client on your machine opens a persistent connection to the PortShare server. The server gives you a public URL like:

```
https://your-service.portshare.kexoz.dev
```

When someone opens that URL, the server does not host your app. It forwards the HTTP request over the tunnel to your machine, your client hits localhost, and the response is sent back to the visitor.

Pieces of the system

Component	Role
server/ (Go + Gin)	Public reverse proxy, identity DB, WebSocket/SSH tunnels
desktop/ (Electron)	Primary client: claim subdomain, hold tunnel, proxy to localhost
cli/	Headless client with the same WebSocket tunnel protocol
website/ (Next.js)	Marketing / download / pricing - not on the request path

2. High-level architecture

```
Visitor --HTTPS--> PortShare server --WS JSON--> Desktop/CLI --HTTP--> localhost:PORT
                          |
                          +-- Postgres (clientId, subdomain, custom domain, plan)
                          +-- In-memory live tunnel connections
```

There is also an SSH reverse-tunnel path (ssh -R) into the same server. Desktop and CLI use WebSocket; SSH is an alternate transport that still ends at localhost on the user's machine.

3. Connection flow (claim -> connect -> expose)

Step 1 - Identity

The client calls POST /client/identity. If it already has a stored clientId, the server returns that record. Otherwise it creates a new 32-character hex ID (free plan, bandwidth quota). Desktop stores the ID in localStorage; CLI stores it under ~/.portshare/config.json. There is no email login for tunnel ownership - possession of clientId is the credential.

Step 2 - Claim a subdomain

The client checks availability (GET /subdomain/check) then claims (POST /subdomain/claim) with { clientId, subdomain }. Names are lowercase alphanumeric/hyphen, length-limited. Reserved names like "api" are blocked. The mapping is stored in Postgres and is permanent until changed.

Step 3 - Open the tunnel

The client opens a WebSocket to /tunnel/connect?clientId=.... The server registers that socket in an in-memory tunnel store (replacing any older socket for the same ID). Keepalive pings keep the connection healthy; the desktop reconnects with backoff if it drops.

Step 4 - Point at a local port

PUT /client/port stores the chosen port for UI/resume. Actual forwarding uses the port the client holds locally. The server sends each public request to the client; the client fetches http://127.0.0.1:{port} and returns the result.

4. What happens on a public request

When a visitor hits https://{subdomain}.{root}/path:

- Gin receives the HTTP request (NoRoute -> HandlePublicTunnel).
- clientForHost(Host) resolves subdomain or custom domain -> clientId.
- If the tunnel is offline -> error page/JSON.
- Optional gates: rate limit, interstitial cookie, Google auth wall, bandwidth quota.
- Server builds a tunnelRequest JSON message and sends it on the client's WebSocket.
- Client performs the local HTTP call and replies with tunnelResponse.
- Server writes status/headers/body back to the visitor.

WebSocket message shapes

```
# server -> client
{ "id", "method", "path", "headers": { "Name": ["..."] }, "body": "<base64>" }

# client -> server
{ "id", "status", "headers", "body": "<base64>", "error"? }
```

Bodies are base64-encoded. Request bodies are capped (about 10 MiB). If the client does not answer within ~60s, the visitor gets a 504 timeout. Hop-by-hop headers are stripped so the proxy stays well-behaved.

SSH alternate path

With SSH reverse forwarding, the server opens a channel toward the client's forwarded local port and uses a reverse proxy over that channel instead of WebSocket JSON. The public Host -> client mapping is the same idea.

5. Protocols and key code

Layer	Protocol
Visitor -> server	HTTP(S)
Control plane (identity, claim, port)	REST JSON
Desktop / CLI tunnel	WebSocket + JSON request/response
SSH tunnel	SSH tcpip-forward / forwarded-tcpip
Client -> local app	HTTP to 127.0.0.1

Important files

Path	Purpose
server/cmd/api/main.go	Routes, WS upgrade, public proxy, SSH

Path	Purpose
server/internal/services/tunnel.go	ConnectTunnel, HandlePublicTunnel, messages
server/internal/services/client.go	Identity, claim, port, custom domain
server/internal/services/ssh_server.go	SSH reverse tunnels
desktop/frontend/src/lib/tunnel.ts	WS client + local proxy
desktop/electron/src/main.js	Node localhost proxy for packaged app
cli/tunnel/tunnel.go	CLI WS client

6. Permanent vs ephemeral

Permanent (Postgres)	Ephemeral (memory / session)
clientId, subdomain, custom domain	Live WebSocket / SSH connection
Stored port, requireAuth, plan	In-flight request waiters
Bandwidth counters	Reachability (online/offline)

Marketing calls the claimed subdomain a permanent public URL. That is true for the name mapping. The live tunnel itself only works while the desktop/CLI (or SSH session) is connected - though the client reconnects automatically.

7. Custom domains

PUT /client/domain stores a custom hostname. Public traffic is matched by Host header. The product UI expects a CNAME from your domain to {subdomain}.{root}. DNS verification / automatic certificates are infrastructure concerns outside the core tunnel loop - the app logic is Host -> clientId -> tunnel.

8. One-sentence mental model

PortShare is a Host-based reverse proxy that multiplexes public HTTP over a long-lived WebSocket (or SSH) to a client that speaks plain HTTP to localhost - with a durable clientId and subdomain registry in Postgres.

Generated for the PortShare monorepo. Update this doc if tunnel.go / ConnectTunnel contracts change.